# 第39课 动态规划经典题（数字三角形）【选学】

> **本课为选学内容**，旨在为对动态规划感兴趣的同学提供进一步探索的机会。学不懂没关系，能跟着思考就是最大的收获！已经走到第39课的你，本身就非常了不起了！

---

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

## 五、课堂例题

### 例题1：3行小三角形（入门版）

**题目**：给定一个3行的数字三角形，求从顶部到底部的最大路径和。

```
  1
 3 2
4 10 1
```

**讲解思路**：

1. 先把三角形存到数组里：`a[1][1]=1`，`a[2][1]=3, a[2][2]=2`，`a[3][1]=4, a[3][2]=10, a[3][3]=1`
2. 初始值：`dp[3][1]=4, dp[3][2]=10, dp[3][3]=1`
3. 倒推第2行：
   - `dp[2][1] = max(4, 10) + 3 = 13`
   - `dp[2][2] = max(10, 1) + 2 = 12`
4. 倒推第1行：
   - `dp[1][1] = max(13, 12) + 1 = 14`
5. 答案：14，路线 1310

**完整代码**：

```
#include <iostream>
using namespace std;

int main() {
    // 3行的数字三角形，直接写在代码里
    int n = 3;
    int a[4][4] = {};  // 从下标1开始用，多开一点
    int dp[4][4] = {};

    // 存入三角形的数据
    a[1][1] = 1;
    a[2][1] = 3;  a[2][2] = 2;
    a[3][1] = 4;  a[3][2] = 10; a[3][3] = 1;

    // 第一步：初始值——最后一行直接抄过来
    for (int j = 1; j <= n; j++) {
        dp[n][j] = a[n][j];
    }

    // 第二步：从倒数第二行往上推
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            dp[i][j] = max(dp[i + 1][j], dp[i + 1][j + 1]) + a[i][j];
        }
    }

    // 第三步：输出山顶的值，就是答案
    cout << dp[1][1] << endl;

    return 0;
}
```

**输出：** `14`

> **给同学们的话：**这道题只有3行，你甚至可以用手算验证程序的结果对不对。手算和电脑算出一样的答案，说明我们的思路是对的！

---

### 例题2：n行通用数字三角形（标准版）

**题目：**输入一个 n 行的数字三角形（n ≤ 100），第 i 行有 i 个整数。求从顶部到底部的最大路径和。

**输入样例：**

```
4
2
3 4
6 5 7
4 1 8 3
```

**输出样例：**

```
20
```

**讲解思路：**

和例题1思路完全一样，只不过现在需要用 `cin` 读入数据，行数 n 由用户输入。

**DP四步法分析：**

| 步骤 | 内容 |
|---|---|
| 找状态 | `dp[i][j]` = 从第 i 行第 j 列出发到山脚的最大金币数 |
| 找转移 | `dp[i][j] = max(dp[i+1][j], dp[i+1][j+1]) + a[i][j]` |
| 定初值 | `dp[n][j] = a[n][j]`（最后一行直接就是自己） |
| 写代码 | 两层循环，外层从第 n-1 行到第 1 行，内层从第 1 列到第 i 列 |

**完整代码：**

```
#include <iostream>
using namespace std;

int a[105][105];   // 存三角形的数据
int dp[105][105];  // 存DP结果

int main() {
    int n;
    cin >> n;

    // 读入三角形
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cin >> a[i][j];
        }
    }

    // 初始值：最后一行
    for (int j = 1; j <= n; j++) {
        dp[n][j] = a[n][j];
    }

    // 自底向上推
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            dp[i][j] = max(dp[i + 1][j], dp[i + 1][j + 1]) + a[i][j];
        }
    }

    // 输出答案
    cout << dp[1][1] << endl;

    return 0;
}
```

**输出：** `20`

> **代码解读：**
> - 数组开 `105` 是为了保险，比 100 多一点，防止越界。
> - 外层循环 `i` 从 `n-1` 到 `1`，是"自底向上"的体现。
> - 内层循环 `j` 从 `1` 到 `i`，因为第 i 行只有 i 个数。
> - `max()` 函数帮我们挑两条路中金币多的那条——这就是"贪心选择"在DP里的体现。

---

## 附：例题源代码与运行结果

### 例题 1：3-Row Small Triangle (Intro Version)

```
// Given a 3-row number triangle, find the maximum path sum
// from top to bottom using bottom-up DP.
#include <iostream>
using namespace std;

int main() {
    // 3-row number triangle, hardcoded for demonstration
    int n = 3;
    int a[4][4] = {};  // 1-indexed, extra space
    int dp[4][4] = {};

    // Store the triangle data
    a[1][1] = 1;
    a[2][1] = 3;  a[2][2] = 2;
    a[3][1] = 4;  a[3][2] = 10; a[3][3] = 1;

    // Step 1: Initialize — copy the last row
    for (int j = 1; j <= n; j++) {
        dp[n][j] = a[n][j];
    }

    // Step 2: Bottom-up DP
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            dp[i][j] = max(dp[i + 1][j], dp[i + 1][j + 1]) + a[i][j];
        }
    }

    // Step 3: The answer is at the top
    cout << dp[1][1] << endl;

    return 0;
}
```

#### Expected Output

```
14

Path: 1 -> 3 -> 10 = 14
```

## 例题 2：N-Row Number Triangle (Standard Version)

```
// Input an n-row number triangle (n <= 100), find the maximum
// path sum from top to bottom using bottom-up DP.
#include <iostream>
using namespace std;

int a[105][105];   // Triangle data
int dp[105][105];  // DP results

int main() {
    int n;
    cin >> n;

    // Read the triangle
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cin >> a[i][j];
        }
    }

    // Initialize: last row
    for (int j = 1; j <= n; j++) {
        dp[n][j] = a[n][j];
    }

    // Bottom-up DP
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            dp[i][j] = max(dp[i + 1][j], dp[i + 1][j + 1]) + a[i][j];
        }
    }

    // Output the answer
    cout << dp[1][1] << endl;

    return 0;
}
```

### Sample Input

```
4
2
3 4
6 5 7
4 1 8 3
```

### Expected Output

```
20

Path: 2 -> 4 -> 7 -> 8 = 20 (going right-right-left from top)
```